PromptForge

OFFICIAL CAPSTONE DOSSIER · END-TO-END GENERATIVE AI APPLICATION WITH ADVANCED PROMPT ENGINEERING

PromptForge is a production-style generative AI studio that turns prompt engineering from an ad-hoc exercise into a measurable, versioned, evaluated engineering discipline. It ships a complete assistant pipeline - layered prompt architecture, external knowledge retrieval, context memory, A/B experimentation, an automated quality evaluator, graceful fallbacks, and full logging - behind one deployed application.

1. Problem statement

Most LLM prototypes fail for predictable reasons: prompts live nowhere (no versioning), answers are never measured (no evaluation), users receive confident hallucinations (no grounding and no fallback), and conversations forget themselves (no context management). PromptForge solves all five at once:

• Unmanaged prompts - a version store with save, reload and A/B comparison against any prior version.
• Unmeasured answers - a second AI evaluator scores every response (quality /100, relevance, clarity, tone /10) with one-line feedback, aggregated into all-time analytics.
• Confident hallucinations - keyword retrieval from an editable external knowledge base grounds answers; off-scope requests trigger a marked fallback instead of a guess.
• Amnesia - the full conversation history plus explicit memory rules let follow-ups like 'how long until it arrives?' resolve correctly.
• One-off demos - every conversation, message, score and prompt version is logged in a hosted database, so quality is tracked across sessions.

Business use cases are first-class: six ready presets (banking support, HR policy, course advising, e-commerce, travel planning, fitness coaching) each define scope, tone, fallback behaviour, few-shot examples and a knowledge base.

2. Architectural implementation

Request flow (one user turn, server-side)

```
User ▶ Studio UI ▶ server function ▶ 1. log user message ▶ 2. retrieve knowledge (RAG) ▶ 3. build layered prompt + full history ▶ 4. AI model ▶ 5. detect fallback marker ▶ 6. AI evaluator scores the answer ▶ 7. log answer + scores ▶ UI renders badges.
```

The client is a React + TanStack Start application; all AI calls run in server functions so the model key never reaches the browser. The evaluator is a second, independent model call that returns strict JSON, so quality gates are automated rather than eyeballed.

Data model

prompt_versions	Every saved prompt: use case, system prompt, fallback, tone - reloadable for experiments.
conversations	One row per chat, recording which prompt version produced it.
messages	Each turn with role, content, fallback flag, evaluator score and feedback.

Deployment is one click: the same build publishes to a live URL with the hosted database and the AI gateway configured server-side - the complete deployment-ready workflow.

3. Prompt strategies

The layered instruction architecture (sent every turn):

1. Use case - the business scope, injected as the first line of every instruction set.
2. System prompt - the authoritative behaviour specification written by the prompt engineer.
3. Tone requirement - how the assistant must sound.
4. Few-shot examples - 1-2 ideal Q→A demonstrations that condition output style and structure (advanced).
5. Retrieved knowledge - top-matching facts from the external knowledge base, cited as [1] [2] [3] on the answer.
6. Memory rules - use full history to resolve 'it', 'that', 'the last one'; never invent facts.
7. Fallback rule - if out of scope or unreliable, reply with the configured fallback and the exact marker so the system can log it.

User messages always travel separately from instructions, so a user cannot override policy through their own text (prompt-injection containment).

3b. Advanced techniques

• A/B prompt comparison - One identical question fans out to two configurations concurrently; both answers are scored, and the panel declares a winner with a rationale - evidence before you commit a version.
• Few-shot conditioning - The examples field is part of the versioned prompt layer, so style is testable like any other parameter.
• Latency instrumentation - Generation + evaluation time is measured per answer and per A/B side (⚡ badge) - speed is treated as a first-class metric next to quality.
• All-time analytics - Conversations, average quality, fallback rate and a sparkline of recent scores aggregate across every session and prompt version.
• LLM-as-judge evaluation - A strict second model call returns JSON scores; evaluator failures degrade gracefully instead of breaking the chat.

4. Benchmark evaluations (live test results)

Grounded answer	Card-blocking question on the NovaBank preset	**94/100 quality**	SOURCES badge shown; facts drawn from knowledge base
Memory check	Follow-up: 'how long until it arrives?'	**Resolved correctly**	History resolved 'it' to the replacement card
Fallback trigger	Off-topic weather question	**FALLBACK badge**	Marked fallback reply returned instead of a guess
Versioning + A/B	Same question through two saved versions	**Winner declared**	Both sides scored; latency per side
Logging + analytics	All-time metrics panel	**Aggregates verified**	Conversations, avg score, fallback rate, score sparkline

Requirements coverage (all 9)

1 Business use case definition	Six presets + free-text use case, injected as line 1 of every prompt
2 System and user prompt architecture	Seven-layer instruction set; user text kept separate from policy
3 Context management and memory	Full history + memory rules; conversations and messages logged
4 Prompt versioning and experimentation	Save / reload versions; A/B lab with scored winner
5 Response evaluation and quality metrics	LLM-as-judge scores + all-time analytics with fallback rate
6 Error handling and fallback prompts	Marked fallback, FALLBACK badge, toast + message restore on errors
7 Integration with external data sources	Editable knowledge base with per-question retrieval and SOURCES badges
8 Deployment-ready application workflow	Full-stack build, server-held keys, one-click publish
9 Complete project documentation	In-app Docs page mapping every requirement plus this dossier

5. Expected outcomes

✓ End-to-end experience: UI, prompt layer, AI gateway, evaluation loop, storage and deployment in one system.
✓ Industry-grade capstone: versioning, logging, automated QA scoring and grounded answers mirror real production assistants.
✓ Deep prompt-engineering practice: layered instructions, tone control, few-shot conditioning, grounding, explicit fallback markers, LLM-as-judge scoring and A/B testing.
✓ Job relevance: covers the daily work of Prompt Engineers (prompt design & testing), Generative AI Engineers (RAG, pipelines, evaluation) and AI Product Developers (use-case framing, metrics, shipping).

6. Exportable artifacts

• This dossier - problem statement, architecture, strategies, benchmarks and coverage table (PDF).
• Prompt versions - every saved configuration is exportable from the studio as reloadable, comparable variants.
• Conversation logs - full message history with evaluator scores and feedback, available from the hosted database.
• Metrics snapshot - all-time averages, fallback rate and the score sparkline from the quality-metrics panel.